

## DEFINICIÓN DEL PROBLEMA

### 1.1 Descripción General del Caso de Uso

En un entorno de procesamiento masivo o computación en la nube, un servidor cuenta con un tiempo total disponible de operación limitante ( $T$ ). El sistema recibe un conjunto de  $n$  tareas entrantes. Cada tarea  $i$  posee un tiempo de ejecución estimado  $t_i$  y un valor de prioridad o beneficio asociado  $p_i$ .

El objetivo es determinar el subconjunto de tareas que maximice la prioridad total procesada sin que la suma de sus tiempos de ejecución supere la capacidad máxima  $T$  del servidor.

### 1.2 Especificación Formal de Entradas, Salidas y Restricciones

#### Entradas:

- Un entero positivo  $n$  que representa la cantidad total de tareas disponibles.
- Un entero positivo  $T$  que representa la capacidad máxima de tiempo del servidor.
- Una lista de  $n$  elementos, donde cada elemento es una estructura  $Tarea_i = (id_i, t_i, p_i)$  con  $id_i$  de 1 a  $n$ , tiempo  $t_i > 0$  y prioridad  $p_i > 0$ .

#### Salidas:

- Una lista  $S$  con los identificadores de las tareas seleccionadas.
- La prioridad total obtenida: suma de las prioridades de las tareas seleccionadas.
- El tiempo total de procesamiento consumido: suma de los tiempos de las tareas seleccionadas.

#### Restricciones:

- Capacidad del servidor: El tiempo total consumido debe ser menor o igual a  $T$ .
- Indivisibilidad de tareas (Decisión Binaria): Para cada tarea  $i$ , la variable de decisión es  $x_i$  en  $\{0, 1\}$ . No se permiten ejecuciones parciales de una tarea.
- No negatividad:  $t_i > 0$  y  $p_i > 0$  para todas las tareas.

### 1.3 Conjuntos de Datos de Prueba

Para validar y evaluar el algoritmo se diseñaron tres casos de prueba con escala progresiva:

-Caso Pequeño (datos/caso\_pequeno.txt):  $n = 10$  tareas,  $T = 50$  unidades de tiempo. Diseñado para verificar la exactitud de los resultados.

-Caso Mediano (datos/caso\_mediano.txt):  $n = 20$  tareas,  $T = 100$  unidades de tiempo. Diseñado para evaluar el crecimiento del árbol recursivo.

-Caso Grande (datos/caso\_grande.txt):  $n = 28$  tareas,  $T = 150$  unidades de tiempo. Representa el límite operativo práctico para el enfoque de fuerza bruta antes de experimentar tiempos de ejecución excesivos.

## SOLUCIÓN POR FUERZA BRUTA

### 2.1 Estrategia Algorítmica

La solución por Fuerza Bruta explora exhaustivamente el espacio de soluciones evaluando las  $2^n$  combinaciones posibles de tareas. Para cada tarea  $i$ , el algoritmo evalúa de forma recursiva dos ramas de decisión:

Excluir la tarea  $i$ : Se pasa a la siguiente tarea manteniendo la capacidad  $T$  intacta.

Incluir la tarea  $i$ : Si  $t_i$  es menor o igual a la capacidad restante, se descuenta  $t_i$  de  $T$ , se suma su prioridad  $p_i$  y se continúa con el análisis de las tareas restantes.

Al finalizar el recorrido de ambas posibilidades, el algoritmo retorna la combinación que entregó la mayor suma de prioridad respetando la capacidad  $T$ .

### 2.2 Explicación del Código Fuente (src/fuerza\_bruta.py)

El archivo `src/fuerza_bruta.py` contiene la implementación completa estructurada en las siguientes funciones principales:

-`cargar_caso(ruta_archivo)`: Lee los archivos de entrada `.txt`, extrae las variables  $n$  y  $T$ , y construye los objetos de la clase `Tarea`.

-`fuerza_bruta_rec(tareas, capacidad, indice)`: Función recursiva que construye el árbol binario de decisión de profundidad  $n$ . Compara las ramas incluir y excluir retornando el máximo valor y la lista de IDs seleccionados.

-`resolver_fuerza_bruta(ruta_archivo)`: Orquestador que inicia el cronómetro, ejecuta la función recursiva y calcula el tiempo de ejecución empírico.

## ANÁLISIS TEÓRICO DE COMPLEJIDAD

### 3.1 Complejidad Temporal: $O(2^n)$

El análisis de la complejidad temporal se deduce evaluando la cantidad de nodos generados en el árbol de llamadas recursivas:

Modelo de Decisión: En cada nivel del árbol (asociado a una tarea  $i$ ), se generan 2 llamadas recursivas adicionales.

Ecuación de Recurrencia:  $T(n) = 2 * T(n-1) + O(1)$

Expansión del Árbol de Recursión:

-Nivel 0 (Raíz):  $2^0 = 1$  llamada

-Nivel 1:  $2^1 = 2$  llamadas

-Nivel 2:  $2^2 = 4$  llamadas

-Nivel  $k$ :  $2^k$  llamadas

-Nivel  $n$ :  $2^n$  llamadas en las hojas

Cálculo del Total de Operaciones: Nodos Totales = Suma desde  $k=0$  hasta  $n$  de  $(2^k) = 2^{(n+1)} - 1$

Dado que cada nodo realiza un número constante de operaciones de comparación y suma  $O(1)$ , descartando las constantes se concluye que la complejidad temporal en el peor y promedio de los casos es exponencial:  $O(2^n)$ .

## MEDICIÓN EMPÍRICA Y RESULTADOS

### 4.1 Tabla de Tiempos de Ejecución

| Caso de Prueba | Tareas Totales (n) | Capacidad Servidor (T) | Prioridad Maxima Obtenida | Tareas Seleccionadas (IDs) | Tiempo de Ejecucion (segundos) |
|----------------|--------------------|------------------------|---------------------------|----------------------------|--------------------------------|
| Pequeño        | 10                 | 50                     | 90                        | 1, 6, 9, 10                | 0.000000 s                     |
| Mediano        | 20                 | 100                    | 183                       | 2, 6, 7, 9, 12, 16, 19     | 0.061084 s                     |

| Caso de Prueba | Tareas Totales (n) | Capacidad Servidor (T) | Prioridad Maxima Obtenida | Tareas Seleccionadas (IDs)             | Tiempo de Ejecucion (segundos) |
|----------------|--------------------|------------------------|---------------------------|----------------------------------------|--------------------------------|
| Grande         | 28                 | 150                    | 275                       | 2, 6, 7, 9, 16, 18, 22, 23, 24, 25, 26 | 11.215471 s                    |

#### 4.2 Gráfica de Tiempo vs Tamaño de Entrada (n)



#### 4.3 Discusión de Resultados Empíricos

Los resultados obtenidos en las pruebas confirman la hipótesis teórica:

- Para  $n = 10$ , el tiempo de cómputo es insignificante (0.000000 segundos).
- Al aumentar el tamaño a  $n = 20$ , el tiempo es de apenas 0.061084 segundos.
- Al aumentar a  $n = 28$ , la ejecución se eleva drásticamente a 11.215471 segundos.

Este incremento no lineal evidencia la curva exponencial caracterizada por  $O(2^n)$ , demostrando que el enfoque por Fuerza Bruta es inviable para instancias donde  $n \geq 30$ , justificando la necesidad de aplicar paradigmas más eficientes en entregas posteriores.